

Coasterbot

Phasenpaper M0

Gereon Such

Wilhelm Büchner University of Applied Sciences
Projektseminar Master

Coasterbot

Phasenpaper M0



Dennis Borutta, B. Eng	dennis.borutta@outlook.com
Stefan Etringer, B. Eng	mail@stefan.works
Gereon Such, M. Sc.	gereonsuch@gmail.com

Versionshistorie

Version	Datum	Bearbeiter	Änderungen
1	20.07.2026	Such	Erstfassung des Phasenpapiers M0
2	21.07.2026	Etringer	Literaturrecherche zu Algorithmen ergänzt
3	21.07.2026	Borutta, Such	Literaturrecherche und Einleitungstext ergänzt; Coasterbot-Abbildung
4	22.07.2026	Such	Versionshistorie ergänzt
5	23.07.2026	Borutta	Projektsteckbrief in Tabelle umgewandelt. Dokumentenstruktur ang

Projektübersicht

Kategorie	Inhalt
Kategorie	Inhalt
Projektname	Coasterbot – Entwicklung eines Prototyps
Projektzeitraum	Beginn: 15.07.2026 Abschluss: 07.11.2026
Projektziel	Entwicklung eines Prototyps eines Coasterbots: ein handtellergroßer Roboter, der selbstständig auf einem Tisch navigiert, dort Getränkeuntersetzer in Richtung von Personen ausgibt und diese automatisiert wieder aufnimmt.
Weitere Ausbaustufen (Nicht im Fokus)	Automatisierte Tischreinigung, Aufnahme von Bestellungen, Initialisierung von Bezahlvorgängen
Motivation / Nutzen	• Entlastung des Bedienpersonals durch reduzierten Aufgabenbereich unterstützt damit den de seit Jahren bestehenden Fachkräftemangels in der Gastronomie [?] • Alleinstellungsmerkmal und zusätzliches Attraktivitätsmerkmal für Bars/Restaurants • Potenziell mehr positive Bewertungen und verbesserte Online-Präsenz und damit Positiver Einfluss auf den Umsatz (Online-Präsenz als zunehmend relevanter Faktor bei der Restaurantwahl) [?]

Kategorie	Inhalt
Definition of Done	• STL-Dateien und Schaltpläne zur Fertigung eines Prototypen sind fertiggestellt • Softwarearchitektur und -komponenten sind ausgearbeitet • Eine Simulation des Programmablaufes ist erstellt und validiert • Tests, die sich aus den Anforderungen ergeben, sind identifiziert und definiert
Out of Scope	• EMV-Betrachtungen • Kartierung der Umgebung • Einbindung in ein Netzwerk • Marketingkonzepte • Schwarmverhalten mehrerer Coasterbots

Projektorganisation

Das Projektteam besteht aus den drei Mitgliedern

Dennis Burutta	Leiter/Mitglied	dennis.borutta@outlook.com
Stefan Etringer	Admin/Mitglied	mail@stefan.works
Gereon Such	Mitglied	gereonsuch@gmail.com

Für die Projektorganisation und dokumentenbasierte Kommunikation im Team wird das Tool Notion <https://app.notion.com> genutzt. In diesem werden Projektphasen, Arbeitspakete, Fortschritt, Ergebnisse sowie Meilensteine protokolliert. Notion bietet hier vielfältige Planungsmöglichkeiten sowie Visualisierungen dazu. Eine Besonderheit dabei ist, dass Notion die Möglichkeit bietet kollaborativ an Dokumenten zu arbeiten und diese direkt an Arbeitspaketen zu knüpfen. So können die Arbeitsergebnisse und Referenzen strukturiert abgelegt werden.

Entwicklungsrelevante Dateien wie Source Code, CAD-Dateien für das Chassis oder Schaltungen werden in einem gemeinsamen Git Repository abgelegt.

Zur Remote Kommunikation wird eine Kombination aus Instant Messaging für fristige Information und Absprachen sowie regelmäßigen Videocalls mit dem Tool Alfaview <https://alfaview.com> genutzt.

Projektplanung

Mit Hilfe der Tools Draw.io und Notion wird im Team die Projektplanung durchgeführt. Hierzu wird zur Ermittlung notwendiger Teilaufgaben und Arbeitspakete die Kreativmethode der Mind-Map angewandt. Daraus ergibt sich der Projektstrukturplan mit Arbeitspaketen gezeigt. Aus diesem wird anschließend eine Vorgangsliste erstellt, die in einem Netzplan überführt wird. Mit Hilfe des Netzplanes wird anschließend ein detaillierter Zeitplan in Notion erstellt.

Projektstruktur

Zur weiteren Bestimmung von erforderlichen Teilaufgaben und Arbeitspaketen wurde die Kreativmethode der Mind-Map angewandt. Die Ergebnisse hierzu sind in Abbildung 1 zu sehen. Daraus ergibt sich der Projektstrukturplan mit Arbeitspaketen wie in Abbildung 2 gezeigt. Hierbei wird ersichtlich, dass der Strang "Organisation" aus der Mind-Map nicht in dem Projektstrukturplan zu sehen ist. Dies wird begründet, da es sich dabei um fortlaufende Aufgaben handelt, die über die gesamte Projektlaufzeit hinweggeführt werden.

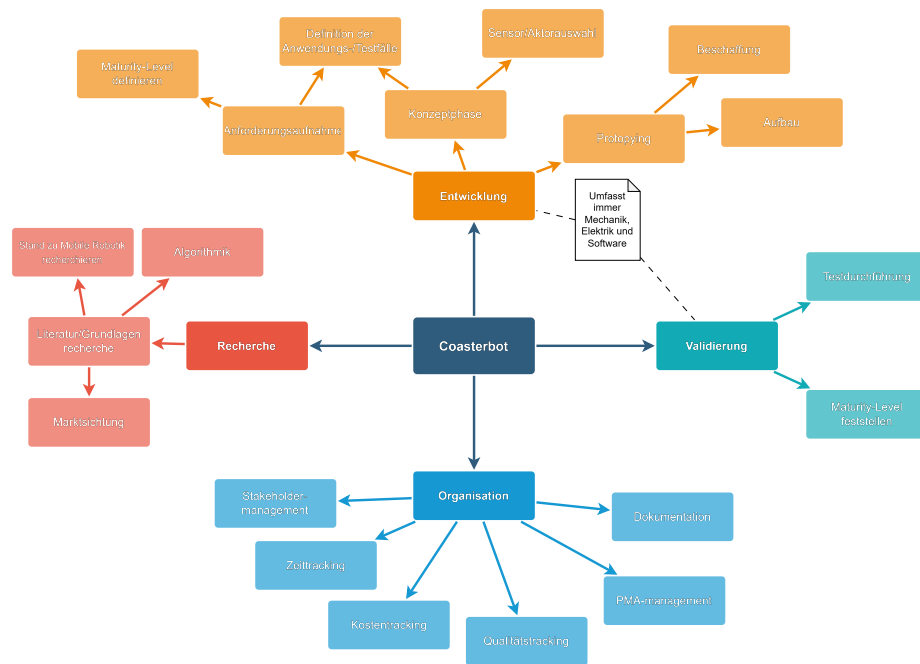


Figure 1: Mindmap zur Projektstruktur

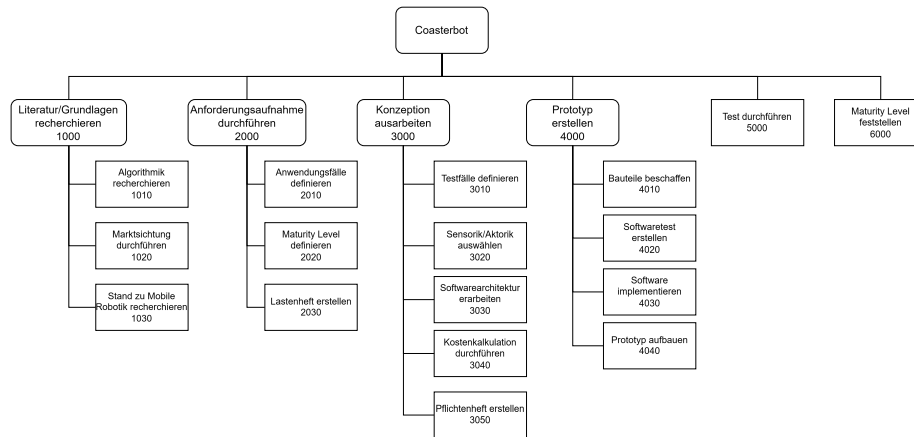


Figure 2: Projektstrukturplan

Netzplan

Mit dem Zeithorizont von 4 Monaten ergibt sich mit Gewichtung der jeweiligen Arbeitspakete nach Aufwand und Priorität der in Abbildung [fig:netzplan] dargestellte Netzplan nach DIN 69900. Aufgrund seiner Breite ist dieser auf einer eigenen Seite im Querformat abgebildet.

Zeitplan

Mit Hilfe des Netzplanes wird der Zeitplan detailliert. Dieser ist in Abbildung[fig:notiontimeline_full] zu sehen. Aufgrund der Größe ist er auf einer eigen Seite im Querformat zu sehen und es sind nur Phase, Teilaufgaben und Meilensteine dargestellt.

Stand der Forschung: Mobile Roboter

Das Feld der mobilen Roboter ist bereits weit erforscht und es lassen sich eine Vielzahl an Arbeiten finden, die unterschiedliche Ansätze und Lösungen für die Navigation, Manipulation und Interaktion von Robotern in verschiedenen Umgebungen präsentieren. Eine übersichtliche Zusammenfassung relevanter Arbeiten ist in der Tabelle 1 im Anhang dargestellt. Diese Tabelle enthält Informationen zu den jeweiligen Roboter-Projekten. Diese werden als Grundlage zur Bauteilauswahl und Softwarearchitektur des Coasterbots dienen. Insbesondere die Arbeit von [?] kann als Grundlage dienen, da hier mehrere mobile Roboter Varianten vorgestellt und mit fertigen Stücklisten und CAD-Dateien veröffentlicht wurden. Damit bieten diese Arbeiten eine gute Grundlage für die Entwicklung des Coasterbots, so dass sich das Team auf die Softwarearchitektur, die Navigation des Roboters und die Hauptfunktion das Ablegen und Aufnehmen

von Getränkeuntersetzter konzentrieren kann.

Stand der Forschung: Algorithmen

Klassische Graphalgorithmen

Die Grundlage vieler Verfahren zur autonomen Navigation bilden klassische Graphalgorithmen. Dabei wird die Umgebung als Graph modelliert, dessen Knoten diskrete Positionen oder Zustände repräsentieren, während Kanten mögliche Bewegungen zwischen diesen beschreiben. Ziel ist die Bestimmung eines optimalen Pfades hinsichtlich einer Kostenfunktion, beispielsweise der zurückgelegten Distanz, Fahrzeit oder des Energieverbrauchs.

Ein Vertreter ist der Dijkstra-Algorithmus, der den kürzesten Pfad in Graphen mit Kantengewichten berechnet. Der Algorithmus garantiert optimale Lösungen insofern die Kantengewichte nicht-negativ sind und stellt ein Referenzverfahren für die globale Pfadplanung in statischen Umgebungen dar [?]. Seine mathematische Grundlage sowie Laufzeitanalyse werden ausführlich in "Introduction to Algorithms" von Cornet et al. beschrieben [?].

Neben Dijkstra existieren weitere grundlegende Suchverfahren, wie die Breiten-suche (Breadth-First Search) und Tiefensuche (Depth-First Search, DFS) [?]. Während BFS optimale Lösungen in ungewichteten Graphen liefert, dient DFS vor allem der Strukturanalyse von Graphen und spielt in der Roboternavigation eine untergeordnete Rolle.

Für Anwendungen mit einer vollständig bekannten und statischen Umgebung stellen diese Algorithmen eine zuverlässige Grundlage der globalen Pfadplanung dar.

Heuristische Verfahren

Da klassische Graphalgorithmen bei großen Suchräumen einen hohen Rechenaufwand verursachen, werden in der mobilen Robotik häufig heuristische Suchverfahren eingesetzt. Ein bedeutendes Verfahren ist der heuristische A* Algorithmus, um die Suche gezielt in Richtung des Zielknotens zu lenken [?, ?].

Die Bewertung eines Knotens erfolgt über

$$f(n) = g(n) + h(n)$$

Hierbei stellen $g(n)$ die wirklichen Kosten eines optimalen Pfades s nach n und $h(n)$ die wirklichen Kosten eines optimalen Pfades von n zu einem präferierten Zielknoten von n darstellt. Allgemein ist $h(n)$ eine Schätzung der verbleibenden Distanz zum Ziel [?]. Ist die Heuristik zulässig (admissible), garantiert A* ebenfalls optimale Lösungen, untersucht jedoch weniger Knoten als Dijkstra.

Für mobile Roboter kommen häufig euklidische oder Manhattan-Distanzen [?] als Heuristik zum Einsatz. Dadurch eignet sich A* insbesondere für Gitternetze (Occupancy Grids), wie sie in der Robotik genutzt werden[?].

Aktuellere Varianten wie Weighted A* oder Anytime A* priorisieren kürzere Rechenzeiten gegenüber der optimalen Lösung, weshalb deren Anwendung geeignet in Echtzeitsystemen ist[?, ?].

Dynamische Pfadplanung

In realen Einsatzgebieten verändert sich die Umgebung häufig während der Fahrt. Statische Algorithmen müssten den gesamten Pfad erneut berechnen. Dies führt zu hohen Rechenzeiten.

Hier setzen Verfahren der dynamischen Pfadplanung an. Zu nennen sind unter anderem D* (Dynamic A*) [?] sowie dessen Weiterentwicklung D* Lite [?]. Beide Verfahren aktualisieren ausschließlich die von einer Umweltveränderung betroffenen Bereiche des Suchgraphen und vermeiden dadurch vollständige Neuberechnungen.

Ein weiteres relevantes Verfahren ist Lifelong Planning A* (LPA*), welches inkrementelle Änderungen effizient verarbeitet [?]. Solche Algorithmen werden insbesondere in autonomen Fahrzeugen und mobilen Servicerobotern eingesetzt, deren Umgebungen durch Menschen oder bewegliche Objekte beeinflusst werden.

Für einen Coaster Bot wären dynamische Planungsverfahren dann relevant, wenn sich während der Navigation Gläser, Teller oder Personen im Arbeitsbereich befinden.

Kollisionsvermeidung

Während globale Pfadplanung den optimalen Weg bestimmen, übernimmt die Kollisionsvermeidung die kurzfristige Reaktion auf Hindernisse.

Ein Ansatz sind Potentialfeldmethoden, bei denen das Ziel eine anziehende Kraft und Hindernisse abstoßende Kräfte erzeugen. Diese Methoden sind einfach implementierbar, können jedoch in lokalen Minima stecken bleiben [?].

In mobilen Robotersystemen wird u.a. der Dynamic Window Approach (DWA) verwendet, der die Bewegungsdynamik des Roboters selbst einbezieht [?]. Diese Strategie ist grundsätzlich ein statischer Ansatz, der die Eigenschaften von Veränderungen in der Umgebung nicht berücksichtigt. Durch den Einsatz von Neuronalen Netzen kann DWA, durch kurze Observationen der sich verändernden Umgebung, die Bewegungen der Hindernisse modellieren und somit auf Veränderungen der Umwelt reagieren [?].

Für komplexe Konfigurationsräume kommen außerdem samplingbasierte Verfahren wie Rapidly-exploring Random Trees (RRT) [?] sowie RRT* [?] zum

Einsatz. Während RRT schnell zulässige Pfade findet, verbessert RRT* diese schrittweise bis zum asymptotischen Optimum [?].

Somit bildet die Kombination aus globalem Pfadplaner und lokaler Kollisionsvermeidung einen möglichen Ansatz zur Bewegung des Coaster Bots dar.

SLAM und Lokalisierung

Zur verbesserten Navigation kann ein Roboter seine eigene Position bestimmen und gleichzeitig eine Karte seiner Umgebung aufbauen. Dieses Problem wird als Simultaneous Localization and Mapping (SLAM) bezeichnet [?].

Zu den klassischen Verfahren gehören Extended Kalman Filter (EKF-SLAM), FastSLAM auf Basis von Paketfiltern sowie graphbasierte SLAM Verfahren [?, ?].

Besonders graphbasierte Optimierungsverfahren haben sich aufgrund ihrer hohen Genauigkeit etabliert und bilden die Grundlage von Robotik-Frameworks wie ROS2 [?].

Für den Coaster Bot hängt die Bedeutung von SLAM vom Einsatzszenario und den verwendeten Sensoren ab. Wird ausschließlich auf einem bekannten Tisch gearbeitet, genügt meist eine vorher definierte Karte. Soll das System autonom unbekannte Arbeitsbereiche erfassen oder sich frei im Raum bewegen können, ist ein SLAM Verfahren vorteilhaft.

Manipulationsplanung

Neben der Navigation stellt die Manipulationsplanung einen wesentlichen Bestandteil autonomer Serviceroboter dar. Sie umfasst die Planung und Steuerung von Greifbewegungen sowie das sichere Aufnehmen und Ablegen von Objekten. Hierbei müssen sowohl die Kinematik des Roboters als auch mögliche Kollisionen zwischen Greifer, Objekt und Umgebung berücksichtigt werden [?, ?].

Zur Bewegungsplanung von Roboterarmen werden häufig samplingbasierte Verfahren wie RRT [?], RRT* [?] oder Probabilistic Roadmaps (PRM) [?] eingesetzt. Moderne Robotik-Frameworks wie MoveIt 2 [?] kombinieren diese Planungsverfahren mit Kollisionsprüfungen und inverser Kinematik.

Für den Coaster Bot besteht die Manipulationsaufgabe darin, Getränkdeckel präzise aufzunehmen und an einer definierten Zielposition abzulegen. Neben einer genauen Positionierung des Roboters müssen dabei Greifstrategie, Objektorientierung sowie eine zuverlässige Ablage berücksichtigt werden.

Inertialnavigation

Die Inertialnavigation (Inertial Navigation System, INS) beschreibt Verfahren zur Bestimmung der Position, Orientierung und Bewegung eines mobilen Systems

ausschließlich auf Grundlage von Trägheitssensoren. Im Gegensatz zu kamerabasierten oder laserbasierten Lokalisierungsmethoden ist die Inertialnavigation nicht auf externe Referenzpunkte oder eine Sichtverbindung zur Umgebung angewiesen. Dadurch eignet sie sich insbesondere für Anwendungen, in denen optische Sensoren nur eingeschränkt eingesetzt werden können oder zeitweise keine zuverlässigen Umgebungsinformationen zur Verfügung stehen [?, ?].

Ein Inertialnavigationssystem besteht zumeist aus einer Inertial Measurement Unit (IMU), die Beschleunigungssensoren (Accelerometer) und Drehratensensoren (Gyroskope) kombiniert. Moderne Systeme integrieren zusätzlich Magnetometer zur Bestimmung der absoluten Orientierung. Aus den gemessenen Beschleunigungen und Winkelgeschwindigkeiten werden durch numerische Integration Geschwindigkeit, Orientierung und Position des Roboters berechnet. Die mathematischen Grundlagen dieser Verfahren basieren auf den Bewegungsgleichungen der klassischen Mechanik sowie auf Verfahren zur Schätzung von Zustandsgrößen [?, ?].

Ein Problem der Inertialnavigation stellt der Drift dar. Geringe Messfehler der Sensoren werden durch die fortlaufende Integration akkumuliert und führen mit zunehmender Betriebsdauer zu wachsenden Positions- und Orientierungsfehlern. Aus diesem Grund wird eine reine Inertialnavigation in der mobilen Robotik nur selten dauerhaft eingesetzt. Stattdessen erfolgt eine Kombination mit externen Sensorsystemen wie LiDAR, Kameras oder Radencodern, um die Positionsschätzung regelmäßig zu korrigieren [?, ?].

Zur Fusion verschiedener Sensordaten kommen häufig probabilistische Schätzverfahren wie der Kalman-Filter, der Extended Kalman Filter (EKF) [?] oder der Unscented Kalman Filter (UKF) [?] zum Einsatz. Diese Verfahren kombinieren die Messdaten der IMU mit absoluten Positionsinformationen anderer Sensoren und ermöglichen dadurch eine robustere Lokalisierung. In aktuellen Robotersystemen wird zudem vermehrt auf graphbasierte Optimierungsverfahren und faktorgraphbasierte Ansätze zurückgegriffen, welche mehrere Sensordatenquellen gleichzeitig berücksichtigen und die Genauigkeit der Positionsbestimmung verbessern [?, ?].

Die Inertialnavigation findet in zahlreichen Bereichen der Robotik Anwendung. Autonome Serviceroboter, fahrerlose Transportsysteme (AGV), Drohnen sowie mobile Manipulatoren nutzen IMUs zur Stabilisierung der Bewegung, zur Schätzung der Fahrzeuglage und zur kurzfristigen Positionsbestimmung. Besonders in Innenräumen, in denen satellitengestützte Navigationssysteme wie GPS nicht verfügbar sind, stellt die IMU eine wichtige Informationsquelle für die Bewegungsregelung dar [?]. Auch in IoT- und Embedded-Systemen werden aufgrund der geringen Baugröße, des niedrigen Energieverbrauchs und der geringen Kosten häufig kompakte Micro-Electro-Mechanical Systems Inertial Measurement Units (MEMS-IMUs) eingesetzt [?, ?].

Für den geplanten Coaster Bot kann die Inertialnavigation die globale Pfadplanung und sensorbasierte Navigation ergänzen. Während Verfahren wie A* oder

Dijkstra den optimalen Fahrweg berechnen und LiDAR- oder Kamerasysteme Hindernisse erkennen, liefert die IMU kontinuierlich Informationen über Beschleunigung und Drehrichtung des Roboters [?, ?]. Dadurch können Fahrbewegungen zwischen Sensormessungen präziser geschätzt sowie kurzfristige Positionsänderungen erfasst werden. Beim Anfahren definierter Ablage- oder Aufnahmepositionen verbessert die Orientierungsschätzung die Wiederholgenauigkeit des Systems. Die Kombination einer IMU mit Verfahren der globalen Pfadplanung entspricht dem in der mobilen Robotik etablierten Ansatz einer mehrstufigen Navigation [?, ?].

Da sich der Coaster Bot auf einer vergleichsweise kleinen Arbeitsfläche bewegt und Bewegungen präzise erfolgen sollten, ist eine alleinige Inertialnavigation aufgrund der Drift möglicherweise nicht ausreichend. Eine mögliche Kombination aus mehreren Sensoren wie IMU, Radencodern, einem LiDAR- oder kamerabasierten Lokalisierungssystem würden eine effizientere Positionsbestimmung erreichen.

Literaturverzeichnis

Roboter / Paper	Kategorie	Komponente
Roboter / Paper	Kategorie	Komponente
	Mechanischer Aufbau	Leichtbau-Chassis
	Mechanischer Aufbau	Räder & Radstand
	Aktor	2x DC-Motor mit Untersetzungsgetriebe
	Aktor	H-Brücken-Motortreiber
	Sensor	5x IR-Nahbereichssensor
	Sensor	Magnetischer Encoder
	Sensor	Spannungsteiler (Batterieüberwachung)
	Steuerung	ATmega328 Mikrocontroller
	Steuerung	Erweiterungsmodule (Raspberry Pi Zero, RF, WiFi, XBee)
	Software	Arduino IDE (C/C++)
	Software	Stage-Simulator
	Mechanischer Aufbau	CAD-Chassis-Vollmodell
	Mechanischer Aufbau	Antriebsrad-Befestigung
	Aktor	DC-Motor (radachsnah montiert)
	Sensor	– (nicht spezifiziert)
	Steuerung	– (nicht spezifiziert)
	Software	CAD/Solid-Modeling-Software
	Mechanischer Aufbau	3D-Chassis-Modell
	Aktor	2x 12V-DC-Motor mit Getriebekopf
	Sensor	Tachometer/Drehzahlgeber
	Steuerung	PID-Regler

Roboter / Paper	Kategorie	Komponente
	Software	MATLAB/Simulink
	Mechanischer Aufbau	Rad-Chassis (Kit-Basis)
	Aktor	DC-Antriebsmotoren
	Sensor	IR-LED + IR-Fototransistor
	Steuerung	Arduino-Board
	Software	Arduino IDE, Schwellenwertlogik
	Mechanischer Aufbau	3D-gedrucktes Chassis (9 Teile)
	Mechanischer Aufbau	O-Ring-Reifen & M3-Verschraubung
	Aktor	2x modifizierter Hobby-Servomotor (Futaba 3003)
	Sensor	2x SRF02-Ultraschallsensor
	Steuerung	PIC16F876A Mikrocontroller (Skycube-Board)
	Software	SDCC Cross-Compiler, OpenSCAD/FreeCAD, KiCad
	Mechanischer Aufbau	Dagu Rover 5 Kettenfahrwerk
	Mechanischer Aufbau	3D-gedruckter Aufbaukörper
	Aktor	Sparkfun Dual-Motortreiber TB6612FNG
	Aktor	Servomotor (rotierbarer Sensorkopf)
	Sensor	3x Devantech SRF-02 + 1x SRF-08 Ultraschallsensor
	Sensor	Devantech CMPS10 Kompass
	Sensor	2x Kamera + GPS-Empfänger
	Steuerung	BeagleBoard-xM (ARM+DSP)
	Steuerung	Pololu Power-Modul, Level-Shifter (Adafruit/Sparkfun)
	Software	On-/Offboard-Softwarestack (Git), Video-Streaming, OpenGL-C
	Mechanischer Aufbau	Zylindrisches Chassis
	Aktor	2x Schrittmotor mit Planetengetriebe
	Sensor	8x IR-Näherungs-/Lichtsensoren (TCRT1000)
	Sensor	3D-Beschleunigungssensor (IMU)
	Sensor	VGA-Farbkamera (640x480)
	Sensor	3x omnidirektionales Mikrofon
	Steuerung	dsPIC-Mikrocontroller
	Steuerung	Bluetooth/RS232-Kommunikation
	Software	GNU GCC C-Compiler, Webots-Simulator
	Mechanischer Aufbau	modulare 3D-druckbare Baugruppen
	Mechanischer Aufbau	CPM-Chassis mit Wechselabdeckung
	Aktor	Antriebsmotoren (Ackermann-Lenkung, Differential- & Mecanum)
	Aktor	-DoF Manipulator-Modul (AMM)
	Aktor	Motortreiber (auf Custom-PCB)
	Sensor	3x Ultraschallsensor (Hinderniserkennung)
	Sensor	IMU (Inertial Measurement Unit)
	Sensor	Motor-Encoder
	Sensor	Erweiterungs-Mounts (LiDAR, Kamera)
	Steuerung	Arduino R3 Mikrocontroller (je Fahrmodul)
	Steuerung	NVIDIA Jetson Nano (CPM)
	Steuerung	Standardisierter I2C-Bus über DB9-Steckverbinder
	Steuerung	3x 18650 Li-Ion-Akku + Wippschalter/Relais

Roboter / Paper	Kategorie	Komponente
	Software	Teleoperations- & Navigationssoftware, Open-Source-Repository